EFFECTIVE JAVA

Item 13
Override clone judiciously

Chapter 3 · Methods common to all objects

WHAT DOES COPYING MEAN?

01 Cloneable and the clone contract

02 PhoneNumber and inheritance

03 Stack: array and elements

04 HashTable: node chains

05 Implementation constraints

06 Copy constructors and factories

CLONEABLE DOES NOT DECLARE CLONE

Mixin

Adds a capability.

Comparable enables comparison.

Cloneable

Zero methods.

Object.clone remains protected.

Implementing the marker does not expose a copy operation.

AN INTERFACE WITH AN OPERATION AND A MARKER

contract/CloneableDemo.java

```
public interface Describable {
    String describe();
}
```

contract/CloneableDemo.java

```
public static final class MarkerOnly implements Cloneable {
}
```

contract/CloneableDemoTest.java

```
void mixinExposesOperation() {
    CloneableDemo.Describable value = () -> "Book Club";

    assertThat(value.describe()).isEqualTo("Book Club");
    assertThat(Cloneable.class.getDeclaredMethods()).isEmpty();
}
```

contract/CloneableDemoTest.java

```
void markerDoesNotGrantAccess() throws ReflectiveOperationException {
    var method = Object.class.getDeclaredMethod("clone");
    var value = new CloneableDemo.MarkerOnly();

    assertThatThrownBy(() -> value.getClass().getMethod("clone"))
            .isInstanceOf(NoSuchMethodException.class);
    assertThatThrownBy(() -> method.invoke(value)).isInstanceOf(IllegalAccessException.class);
}
```

THREE CONVENTIONS

x.clone() != x

x.clone().getClass() ==
x.getClass()

x.clone().equals(x)

Another instance · same type · equality depends on equals

contract/PhoneNumberTest.java

```
void inheritedEqualsUsesIdentity() {
    var original = new Stack();

    assertThat(original.clone()).isNotSameAs(original).isNotEqualTo(original);
}
```

PHONENUMBER: SAME VALUES, ANOTHER INSTANCE

Object.clone copies fields
without running constructors.
Primitives: copied values.
References: the same objects.

contract/PhoneNumber.java

```java
@Override
public PhoneNumber clone() {
    try {
        return (PhoneNumber) super.clone();
    } catch (CloneNotSupportedException impossible) {
        throw new AssertionError(impossible);
    }
}
```

contract/PhoneNumberTest.java

```
void clonePreservesValue() {
    var original = new PhoneNumber(313, 414, 533);
    int before = PhoneNumber.constructorCalls();

    PhoneNumber copy = original.clone();

    assertThat(copy).isNotSameAs(original).isEqualTo(original);
    assertThat(copy.getClass()).isEqualTo(original.getClass());
    assertThat(copy.hashCode()).isEqualTo(original.hashCode());
    assertThat(PhoneNumber.constructorCalls()).isEqualTo(before);
}
```

CONSTRUCTING THE BASE CLASS LOSES THE SUBTYPE

new Parent() is still Parent,
even when the original is Child.

The super.clone chain preserves the runtime class.

TWO STRATEGIES IN THE SAME BASE CLASS

inheritance/InheritanceExample.java

```
public InheritanceExample brokenClone() {
    return new InheritanceExample();
}
```

inheritance/InheritanceExample.java

```
@Override
protected InheritanceExample clone() throws CloneNotSupportedException {
    return (InheritanceExample) super.clone();
}
```

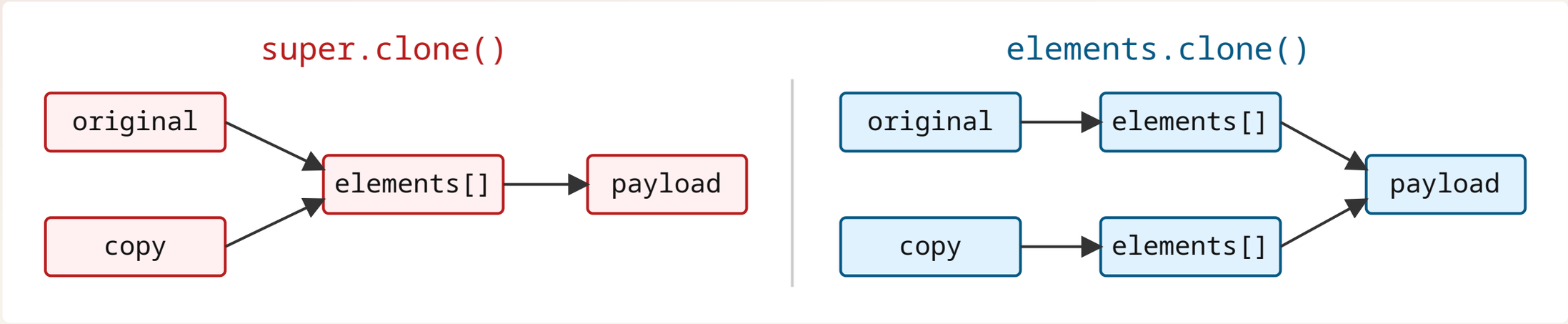
THE TEST: THE CLONE REMAINS CHILD

inheritance/InheritanceExampleTest.java

```
void superClonePreservesSubtype() {
    var original = new InheritanceExample.Child();

    assertThat(original.brokenClone()).isExactlyInstanceOf(InheritanceExample.class);
    assertThatThrownBy(() -> InheritanceExample.Child.class.cast(original.brokenClone()))
            .isInstanceOf(ClassCastException.class);
    var copy = (InheritanceExample.Child) original.clone();
    assertThat(copy).isNotSameAs(original).isExactlyInstanceOf(original.getClass());
    assertThat(copy.label()).isEqualTo("backend");
}
```

STACK: ANOTHER STACK, THE SAME ARRAY?



Copying the stack does not automatically separate its storage.

mutable/Stack.java

```
public Stack shallowCopy() {
    try {
        return (Stack) super.clone();
    } catch (CloneNotSupportedException impossible) {
        throw new AssertionError(impossible);
    }
}
```

mutable/Stack.java

```
@Override
public Stack clone() {
    Stack copy = shallowCopy();
    copy.elements = elements.clone();
    return copy;
}
```

THE TEST: POP MUST NOT CORRUPT THE ORIGINAL

mutable/StackTest.java

```
void cloneOwnsBackingArray() {
    var original = new Stack();
    original.push("top");
    original.shallowCopy().pop();
    assertThat(original.pop()).isNull();

    original.push("top");
    original.clone().pop();
    assertThat(original.pop()).isEqualTo("top");
}
```

NEW ARRAY.
THE SAME ELEMENTS.

StringBuilder[] copies
references to the builders.
To isolate their state, copy each mutable element too.

THE TEST: COPYING THE ARRAY AND COPYING ITS ELEMENT

mutable/StackTest.java

```
void arrayCloneSharesElements() {
    StringBuilder[] original = {new StringBuilder("A")};
    StringBuilder[] copy = original.clone();
    copy[0].append("B");
    assertThat(copy).isNotSameAs(original);
    assertThat(copy[0]).isSameAs(original[0]);
    assertThat(original[0].toString()).isEqualTo("AB");

    StringBuilder[] independent = {new StringBuilder(original[0])};
    independent[0].append("C");
    assertThat(original[0].toString()).isEqualTo("AB");
    assertThat(independent[0].toString()).isEqualTo("ABC");
}
```

HASHTABLE: NEW BUCKETS, SHARED NODES

Copying the bucket array does not copy the nodes in its chains.

The correction must rebuild every node.

AN ITERATIVE COPY OF EACH CHAIN

mutable/HashTable.java

```
private Entry copyChain() {
    Entry head = new Entry(key, value, null);
    Entry tail = head;
    for (Entry node = next; node != null; node = node.next) {
        tail.next = new Entry(node.key, node.value, null);
        tail = tail.next;
    }
    return head;
}
```

mutable/HashTable.java

```
@Override
public HashTable clone() {
    HashTable copy = shallowBucketsCopy();
    for (int i = 0; i < buckets.length; i++) {
        copy.buckets[i] = buckets[i] == null ? null : buckets[i].copyChain();
    }
    return copy;
}
```

mutable/HashTableTest.java

```
void cloneOwnsChain() {
    var original = new HashTable();
    original.put("Aa", "first");
    original.put("BB", "second");
    assertThat("Aa".hashCode()).isEqualTo("BB".hashCode());
    original.shallowBucketsCopy().put("Aa", "shared");
    assertThat(original.get("Aa")).isEqualTo("shared");

    var copy = original.clone();
    copy.put("Aa", "copy tail");
    copy.put("BB", "copy head");
    assertThat(original.get("Aa")).isEqualTo("shared");
    assertThat(original.get("BB")).isEqualTo("second");
    assertThat(copy.get("Aa")).isEqualTo("copy tail");
    assertThat(copy.get("BB")).isEqualTo("copy head");
}
```

PRESERVE THE INVARIANTS

Final references

final does not freeze a list.
clone cannot replace that field.

Checked exception

Without Cloneable, Object.clone throws CloneNotSupportedException.

contract/CloneableDemoTest.java

```
void finalRepairDoesNotCompile() throws IOException {
    assertThat(compile("FinalRepair")).contains("compiler.err.cant.assign.val.to.var");
}
```

contract/CloneableDemoTest.java

```
void missingMarkerRejectsClone() {
    assertThatThrownBy(CloneableDemo::unsupportedCopy)
            .isInstanceOf(CloneNotSupportedException.class);
}
```

CONSTRUCT THE COPY EXPLICITLY

Copy constructor

new Account(original)

Copy factory

Account.copyOf(original)

Normal construction, final fields and a class-defined copy policy.

INITIALIZE A NEW LIST WHILE RETAINING FINAL

alternatives/Account.java

```
private final List<String> labels;

public Account(List<String> labels) {
    this.labels = new ArrayList<>(labels);
}

public Account(Account source) {
    this(source.labels);
}

public static Account copyOf(Account source) {
    return new Account(source);
}
```

THE TEST: EACH COPY OWNS ITS LIST

alternatives/AccountTest.java

```
void copiesOwnFinalList() {
    var original = new Account(List.of("primary"));
    var constructorCopy = new Account(original);
    var factoryCopy = Account.copyOf(original);
    constructorCopy.addLabel("constructor");
    factoryCopy.addLabel("factory");

    assertThat(original.labels()).containsExactly("primary");
    assertThat(constructorCopy.labels()).containsExactly("primary", "constructor");
    assertThat(factoryCopy.labels()).containsExactly("primary", "factory");
    assertThat(factoryCopy).isNotSameAs(original);
}
```

ITEM 13

- ✓ Cloneable does not expose a public operation
- ✓ Object.clone makes a shallow copy
- ✓ Copy mutable state that requires independence
- ✓ Preserve invariants and runtime class
- ✓ Favor constructors and factories in new models

DEFINE THE COPY.

What must be preserved?

What needs to be independent?